

component's `currentState` either internally or from the component instance parent object. Remember that code within a custom component uses this to refer to the current instance of the component. The following code switches the `currentState` of the component instance to a new state:

- `this.currentState = 'myNewState';`

Using transitions

Transitions are a way of associating animations, implemented as Flex effects, with runtime changes from one view state to another. By default, when you switch to a view state that changes the visibility, size, or position of objects on the screen, the change is visually abrupt. A transition enables you to slow down and choreograph the change so that it's easier and more fun to watch. As with view states, you typically declare transitions using MXML code. Each visual component has a `transitions` property. The `transitions` property is an Array containing one or more instances of the `Transition` class. To declare transitions in MXML, you create an `<s:transitions>` tag set as a direct child of the main application or component file's root element, often right after the `<s:states>` declaration. Then nest as many `<s:Transition>` tag sets within `<s:transitions>` as you need:

- `<s:Application...>`
- `<s:states>`
- `...declare <s:State> elements here...`
- `</s:states>`
- `<s:transitions>`
- `...declare <s:Transition> elements here...`
- `</s:transitions>`
- `... declare visual objects here ...`
- `</s:Application>`

In the Flex 4.5 SDK, the `State` and `Transition` classes are still members of the MX package `mx.states`, so they're sometimes declared with an `mx` prefix. However, the Flex SDK declares these classes in both the MX and Spark namespace manifests, so you can use either namespace prefix. Each transition is declared as an `<s:Transition>` tag with the properties `'fromState'`, the name of the starting state and `'toState'` the name of the ending state. Each of these properties can be set to either an explicit state name or a wildcard (*), the default for both properties, to indicate that the transition applies to all state changes. You then indicate which animation you want to play by nesting the appropriate effect class within the `<s:Transition>` tag set. The effect should have its `target` or `targets` property set to indicate which objects should be animated. This transition is applied by